WEEK 2

Null Safety, Collections & Lambdas

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

Last week you learned to work with **single values**. This week you will learn to work with **groups of values** (lists, sets and maps), to handle **missing values** safely (null safety), and to pass **functions as values** (lambdas). You'll use these every day in Android: a list of messages, a map of settings, a click handler written as a lambda.

Day	Topic	You will be able to...
Day 1	Null safety	Use ?, ?., ?:, !! and let, and avoid crashes
Day 2	Lists	Store many values in order, then add, remove, search and sort them
Day 3	Sets & Maps	Store unique values and key-value pairs
Day 4	Lambdas & higher-order functions	Write small functions as values and pass them around
Day 5	Collection operations	Use filter, map, forEach, sortedBy, groupBy and more
Day 6	Weekly project	Build a Student Marks Manager

Before you start

- Finish the Week 1 checklist first. This week builds on variables, if/when, loops and functions.
- Use the **Kotlin Playground** for Days 1 to 5. Use **IntelliJ** for the interactive version of the Day 6 project.
- Study each day in the same 4-hour pattern: **Learn, Practice, Build, Revise**.

> **Tip:** Every example in this book has been run, and the output shown is the real output.

WEEK 2 • DAY 1

Null Safety

Today's goal: Understand what null means, why it causes crashes, and how Kotlin protects you from them.

1. What is null?

null means **"no value"**. For example, a user might not have entered a middle name, or a search might find nothing. In many languages, using a null value by mistake crashes the app (the famous *NullPointerException*). Kotlin prevents most of these crashes **before** your program even runs.

2. Nullable and non-nullable types

By default, a Kotlin variable **cannot** hold null. To allow null, add a **?** after the type.

Type	Can it be null?	Example
String	No	val name: String = "Anjana"
String?	Yes	val middleName: String? = null
Int?	Yes	val score: Int? = null

This gives an ERROR

```
val name: String = null      // Error: Null can not be a value of a non-null type String
```

Nullable variables

```
fun main() {
    var nickname: String? = null
    println(nickname)

    nickname = "Anju"
    println(nickname)
}
```

Output

```
null
Anju
```

You can't use a nullable value directly, because it might be null:

This gives an ERROR

```
val nickname: String? = "Anju"
println(nickname.length)     // Error: Only safe (?.) or non-null asserted (!!.) calls are allowed
```

Kotlin gives you four tools to handle this safely. Let's learn each one.

3. Tool 1: check with if

After you check that a value is not null, Kotlin **remembers** it inside the if block. This is called a **smart cast**.

Null check with if

```
fun main() {
    val nickname: String? = "Anju"

    if (nickname != null) {
        println("Nickname has ${nickname.length} letters")   // safe here
    } else {
        println("No nickname")
    }
}
```

Output

```
Nickname has 4 letters
```

4. Tool 2: the safe call operator ?.

a?.b means: *if a is not null, use b; otherwise the result is null*. It never crashes.

Safe calls

```
fun main() {
    val city: String? = "Kochi"
    val village: String? = null

    println(city?.length)          // 5
    println(village?.length)       // null (no crash)
    println(city?.uppercase())     // KOCHI
    println(village?.uppercase())  // null
}
```

Output

```
5
null
KOCHI
null
```

5. Tool 3: the Elvis operator ?:

a ?: b means: *use a if it is not null, otherwise use b*. Use it to give a **default value**. (Turn your head sideways and ?: looks like Elvis Presley's hair!)

Default values with ?:

```
fun main() {
    val savedName: String? = null
    val displayName = savedName ?: "Guest"
    println("Welcome, $displayName")

    val village: String? = null
    val length = village?.length ?: 0
    println("Length: $length")

    val input = "abc"
    val number = input.toIntOrNull() ?: 0
    println("Number: $number")
}
```

Output

```
Welcome, Guest
Length: 0
Number: 0
```

Tip: ?. and **?:** together are the most common pattern in Android code: `user?.name ?: "Unknown"`.

6. Tool 4: the not-null assertion !!

a!! tells Kotlin: *trust me, this is not null*. If you're wrong, the program **crashes**.

Dangerous: this CRASHES

```
val village: String? = null
println(village!!.length)    // NullPointerException: the app crashes
```

Careful: Avoid **!!**. Professional developers almost never use it. Use if checks, ?. or ?: instead.

7. let: run code only when a value is not null

a?.let { ... } runs the code inside { } only if a is not null. Inside the braces, the value is called **it**.

Using let

```
fun main() {
    val email: String? = "anjana@example.com"
    val phone: String? = null

    email?.let {
        println("Sending mail to $it")
    }
    phone?.let {
        println("Calling $it")      // does not run, phone is null
    }
    println("Done")
}
```

Output

```
Sending mail to anjana@example.com
Done
```

8. Functions with nullable parameters and results

Nullable in functions

```
fun findStudent(rollNumber: Int): String? {
    return when (rollNumber) {
        1 -> "Anjana"
        2 -> "Rahul"
        else -> null          // not found
    }
}

fun greet(name: String?) {
    println("Hello, ${name ?: "stranger"}!")
}

fun main() {
    greet(findStudent(1))
    greet(findStudent(5))

    val student = findStudent(2)
    println("Name length: ${student?.length ?: 0}")
}
```

Output

```
Hello, Anjana!
Hello, stranger!
Name length: 5
```

9. Summary

Syntax	Meaning	Crashes?
String?	This variable may hold null	-
if (x != null)	Check, then use x safely inside the block	Never
x?.length	Use x if it's not null, otherwise give null	Never
x ?: default	Use x if it's not null, otherwise use default	Never
x?.let { }	Run the block only if x is not null	Never
x!!	Force: assume x is not null	**Yes**, if x is null

Practice Exercises

1. Create a nullable String variable **middleName** set to null. Print "No middle name" if it is null, or the name if it is not, using an if check.

2. Using ?. and ?:, print the length of a nullable String, or 0 if it is null. Test it with "Kotlin" and with null.

3. Convert the texts "42", "hello" and "7" to Int using toIntOrNull() and ?: so that invalid text becomes -1. Print all three.

4. Write a function **getGrade(marks: Int?): String** that returns "Absent" if marks is null, otherwise "Pass" (40 or more) or "Fail".

5. Use **let** to print "Phone: ..." only when a nullable phone number is not null. Test it with a number and with null.

6. Write a function **findCapital(state: String): String?** for Kerala, Tamil Nadu and Karnataka, returning null for other states. Print each result with ?: "Unknown".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does null mean?

Q2. How do you make a variable able to hold null?

Q3. What does name?.length give if name is null?

Q4. What does x ?: 10 give if x is null?

Q5. Why should you avoid !! ?

Q6. Inside x?.let { }, what is the value called?

WEEK 2 • DAY 2

Lists: Storing Many Values

Today's goal: Create read-only and mutable lists, read items by index, add and remove items, and loop over them.

1. Why lists?

Imagine storing the marks of 60 students. You don't want 60 separate variables! A **List** stores many values **in order** in one variable. Lists are everywhere in Android: a list of chats, songs, products or contacts.

2. Creating a list

Your first list

```
fun main() {
    val fruits = listOf("Mango", "Banana", "Jackfruit", "Papaya")
    val marks = listOf(85, 92, 78, 66)

    println(fruits)
    println(marks)
    println("Number of fruits: ${fruits.size}")
}
```

Output

```
[Mango, Banana, Jackfruit, Papaya]
[85, 92, 78, 66]
Number of fruits: 4
```

The type of **fruits** is **List<String>**, meaning "a list of Strings". The type in < > tells Kotlin what kind of items the list holds.

3. Reading items by index

Each item has a position called an **index**. Like Strings, indexes start at **0**.

Index	0	1	2	3
Item	Mango	Banana	Jackfruit	Papaya

Accessing items

```
fun main() {
    val fruits = listOf("Mango", "Banana", "Jackfruit", "Papaya")

    println(fruits[0])                  // first item
    println(fruits[2])
    println(fruits.first())
    println(fruits.last())
    println(fruits.indexOf("Banana"))   // position of Banana
    println(fruits.contains("Apple"))
    println("Mango" in fruits)          // same as contains
    println(fruits.getOrNull(10))       // safe: null instead of crash
}
```

Output

```
Mango
Jackfruit
Mango
Papaya
1
false
true
null
```

Careful: fruits[10] crashes with *IndexOutOfBoundsException* because there is no item at index 10. Use **getOrNull()** when you're not sure the index exists.

4. Looping over a list

Three ways to loop

```
fun main() {
    val fruits = listOf("Mango", "Banana", "Jackfruit")

    // 1. Each item
    for (fruit in fruits) {
        println("I like $fruit")
    }

    // 2. With index
    for (i in fruits.indices) {
        println("$i: ${fruits[i]}")
    }

    // 3. Index and item together
    for ((index, fruit) in fruits.withIndex()) {
        println("${index + 1}. $fruit")
    }
}
```

Output

```
I like Mango
I like Banana
I like Jackfruit
0: Mango
1: Banana
2: Jackfruit
1. Mango
2. Banana
3. Jackfruit
```

5. Mutable lists: lists you can change

listOf() creates a **read-only** list. To add or remove items, use **mutableListOf()**.

Changing a list

```
fun main() {
    val tasks = mutableListOf("Study Kotlin", "Drink water")
    println(tasks)

    tasks.add("Exercise")             // add at the end
    tasks.add(0, "Wake up")           // add at index 0
    println(tasks)

    tasks.remove("Drink water")       // remove by value
    println(tasks)

    tasks.removeAt(0)                 // remove by index
    println(tasks)

    tasks[0] = "Study Android"        // replace an item
    println(tasks)

    println("Tasks left: ${tasks.size}")
    tasks.clear()                     // remove everything
    println("Empty? ${tasks.isEmpty()}")
}
```

Output

```
[Study Kotlin, Drink water]
[Wake up, Study Kotlin, Drink water, Exercise]
[Wake up, Study Kotlin, Exercise]
[Study Kotlin, Exercise]
[Study Android, Exercise]
Tasks left: 2
Empty? true
```

Tip: A **val** mutable list can still change its **contents**. val only means the variable always points to the **same list**.

Starting with an empty list

```
fun main() {
    val evenNumbers = mutableListOf<Int>()   // empty list of Int
    for (i in 1..10) {
        if (i % 2 == 0) evenNumbers.add(i)
    }
    println(evenNumbers)
}
```

Output

```
[2, 4, 6, 8, 10]
```

With an empty list, Kotlin can't guess the type, so you write it yourself: **mutableListOf<Int>()**.

6. Useful list functions

Built-in helpers

```
fun main() {
    val marks = listOf(85, 92, 78, 66, 92)

    println("Sum: ${marks.sum()}")
    println("Average: ${marks.average()}")
    println("Highest: ${marks.max()}")
    println("Lowest: ${marks.min()}")
    println("Sorted: ${marks.sorted()}")
    println("Descending: ${marks.sortedDescending()}")
    println("Reversed: ${marks.reversed()}")
    println("Without duplicates: ${marks.distinct()}")
    println("First 3: ${marks.take(3)}")
    println("Joined: ${marks.joinToString(" | ")}")
}
```

Output

```
Sum: 413
Average: 82.6
Highest: 92
Lowest: 66
Sorted: [66, 78, 85, 92, 92]
Descending: [92, 92, 85, 78, 66]
Reversed: [92, 66, 78, 92, 85]
Without duplicates: [85, 92, 78, 66]
First 3: [85, 92, 78]
Joined: 85 | 92 | 78 | 66 | 92
```

sorted(), **reversed()** and the others return a **new** list. The original list doesn't change.

7. Arrays (briefly)

An **Array** is like a list with a **fixed size**. You can change items, but you can't add or remove any. In Kotlin code, lists are used much more often, but you'll see arrays sometimes.

Arrays

```
fun main() {
    val days = arrayOf("Mon", "Tue", "Wed")
    days[1] = "Tuesday"
    println(days.size)
    println(days.joinToString())
    println(days.contentToString())
}
```

Output

```
3
Mon, Tuesday, Wed
[Mon, Tuesday, Wed]
```

Careful: println(days) prints a strange code like *[Ljava.lang.String;@1b6d3586* instead of the items. Use **joinToString()** or **contentToString()** to print an array.

8. Complete example: Shopping List

Shopping list

```
fun main() {
    val items = mutableListOf("Rice", "Milk", "Eggs")
    val prices = mutableListOf(60.0, 28.0, 72.0)

    items.add("Bread")
    prices.add(40.0)

    println("--- Shopping List ---")
    for (i in items.indices) {
        println("${i + 1}. ${items[i]} - Rs ${prices[i]}")
    }
    println("Total items: ${items.size}")
    println("Total cost : Rs ${prices.sum()}")
    println("Costliest  : ${items[prices.indexOf(prices.max())]}")
}
```

Output

```
--- Shopping List ---
1. Rice - Rs 60.0
2. Milk - Rs 28.0
3. Eggs - Rs 72.0
4. Bread - Rs 40.0
Total items: 4
Total cost : Rs 200.0
Costliest  : Eggs
```

Practice Exercises

1. Create a list of your 5 favourite movies. Print the list, its size, and the first and last movie.

2. Create a list of 6 numbers. Print their sum, average, maximum and minimum.

3. Start with an empty mutable list of Strings. Add 4 friends' names, remove one, then print the list with numbering (1., 2., ...).

4. Using a loop, put the squares of 1 to 10 into a mutable list and print it.

5. Given marks = listOf(45, 82, 37, 91, 60), count how many students passed (40 or more) using a for loop.

6. Write a function **secondLargest(numbers: List<Int>): Int** using sortedDescending() and distinct(). Test it with [4, 9, 2, 9, 7].

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between listOf and mutableListOf?

Q2. What is the index of the first item in a list?

Q3. How do you get the number of items in a list?

Q4. What happens if you access an index that doesn't exist?

Q5. Does sorted() change the original list?

Q6. How do you create an empty mutable list of Int?

WEEK 2 • DAY 3

Sets & Maps

Today's goal: Store unique values with Sets and connect keys to values with Maps.

1. Sets: no duplicates allowed

A **Set** is a collection where every item is **unique**. If you add something that's already there, nothing happens.

Creating sets

```
fun main() {
    val colors = setOf("Red", "Green", "Red", "Blue", "Green")
    println(colors)
    println("Unique colors: ${colors.size}")
    println("Has Blue? ${"Blue" in colors}")
}
```

Output

```
[Red, Green, Blue]
Unique colors: 3
Has Blue? true
```

Mutable sets

```
fun main() {
    val visitedCities = mutableSetOf("Kochi", "Munnar")

    println(visitedCities.add("Alappuzha"))   // new: true
    println(visitedCities.add("Kochi"))       // already there: false
    visitedCities.remove("Munnar")

    println(visitedCities)
}
```

Output

```
true
false
[Kochi, Alappuzha]
```

Removing duplicates and comparing sets

```
fun main() {
    val attendance = listOf("Anu", "Ravi", "Anu", "Meera", "Ravi")
    val uniqueStudents = attendance.toSet()
    println(uniqueStudents)

    val kotlinClass = setOf("Anu", "Ravi", "Meera")
    val androidClass = setOf("Ravi", "Meera", "John")

    println("In both: ${kotlinClass intersect androidClass}")
    println("In any: ${kotlinClass union androidClass}")
    println("Only Kotlin: ${kotlinClass subtract androidClass}")
}
```

Output

```
[Anu, Ravi, Meera]
In both: [Ravi, Meera]
In any: [Anu, Ravi, Meera, John]
Only Kotlin: [Anu]
```

Tip: Use a **Set** when you only care **whether** something is present (tags, visited places, unique IDs). Use a **List** when order or duplicates matter.

2. Maps: key and value pairs

A **Map** connects a **key** to a **value**, like a dictionary connects a word to its meaning, or a phone book connects a name to a number. Each key appears only once.

Key	Value
"Kerala"	"Thiruvananthapuram"
"Tamil Nadu"	"Chennai"
"Karnataka"	"Bengaluru"

Creating a map

```
fun main() {
    val capitals = mapOf(
        "Kerala" to "Thiruvananthapuram",
        "Tamil Nadu" to "Chennai",
        "Karnataka" to "Bengaluru"
    )

    println(capitals)
    println(capitals["Kerala"])         // get value by key
    println(capitals["Goa"])            // key not found: null
    println(capitals.getOrDefault("Goa", "Not found"))
    println("Has Kerala? ${capitals.containsKey("Kerala")}")
    println("Size: ${capitals.size}")
}
```

Output

```
{Kerala=Thiruvananthapuram, Tamil Nadu=Chennai, Karnataka=Bengaluru}
Thiruvananthapuram
null
Not found
Has Kerala? true
Size: 3
```

"Kerala" to "Thiruvananthapuram" creates one key-value pair. **map[key]** returns a **nullable** value, because the key might not exist. Use what you learned on Day 1 (?: and let) to handle it.

3. Looping over a map

Keys, values and entries

```
fun main() {
    val marks = mapOf("Maths" to 90, "Physics" to 85, "English" to 78)

    for ((subject, mark) in marks) {
        println("$subject: $mark")
    }
    println("Subjects: ${marks.keys}")
    println("Marks: ${marks.values}")
    println("Total: ${marks.values.sum()}")
}
```

Output

```
Maths: 90
Physics: 85
English: 78
Subjects: [Maths, Physics, English]
Marks: [90, 85, 78]
Total: 253
```

4. Mutable maps

Adding, updating and removing

```
fun main() {
    val phoneBook = mutableMapOf("Amma" to "98470 11111")

    phoneBook["Achan"] = "98470 22222"     // add
    phoneBook["Rahul"] = "90000 33333"
    phoneBook["Amma"] = "98470 99999"      // update (key exists)
    phoneBook.remove("Rahul")              // remove

    for ((name, number) in phoneBook) {
        println("$name -> $number")
    }
}
```

Output

```
Amma -> 98470 99999
Achan -> 98470 22222
```

5. A classic program: counting with a map

Word counter

```
fun main() {
    val sentence = "the cat and the dog and the bird"
    val counts = mutableMapOf<String, Int>()

    for (word in sentence.split(" ")) {
        counts[word] = (counts[word] ?: 0) + 1
    }
    println(counts)
}
```

Output

```
{the=3, cat=1, and=2, dog=1, bird=1}
```

For each word: get its current count (or 0 if it's new, using ?:), add 1 and store it back. **split(" ")** breaks a String into a list of words.

Letter counter

```
fun main() {
    val word = "banana"
    val letterCount = mutableMapOf<Char, Int>()
    for (ch in word) {
        letterCount[ch] = letterCount.getOrDefault(ch, 0) + 1
    }
    println(letterCount)
}
```

Output

```
{b=1, a=3, n=2}
```

6. Which collection should I use?

Collection	Order	Duplicates	Use it for
List	Yes	Allowed	Items in order: messages, songs, marks
Set	Insertion order*	Not allowed	Unique items: tags, visited places
Map	Insertion order*	Keys unique	Looking up a value by key: settings, phone book

** setOf and mapOf keep the order in which you added items.*

Practice Exercises

1. Given the list [3, 7, 3, 9, 7, 1], print the unique numbers using a Set, and how many there are.

2. Create a mutable set of hobbies. Try adding a hobby that's already there and print what add() returns.

3. Create a map of 4 fruits to their prices. Print each fruit and price, and the total price.

4. Create a mutable map of student names to marks. Add a new student, update one student's mark, remove one, then print the map.

5. Count how many times each letter appears in "programming" using a map.

6. Write a function **getMonthDays(month: String): Int** that uses a map of month names to days and returns 0 for unknown months. Test it with "February" and "Funday".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What happens when you add a duplicate item to a Set?

Q2. How do you create one key-value pair for a map?

Q3. What does map[key] return if the key doesn't exist?

Q4. How do you add or update a value in a mutable map?

Q5. How do you convert a List to a Set?

Q6. Can a Map have two equal keys?

WEEK 2 • DAY 4

Lambdas & Higher-Order Functions

Today's goal: Write functions as values (lambdas), store them in variables and pass them to other functions.

1. What is a lambda?

A **lambda** is a small function **without a name** that you can store in a variable or pass to another function. In Android, every button click handler is a lambda: `onClick = { ... }`.

Normal function vs lambda

```
// Normal function
fun square(n: Int): Int {
    return n * n
}

fun main() {
    // The same thing as a lambda stored in a variable
    val squareLambda = { n: Int -> n * n }

    println(square(5))
    println(squareLambda(5))
}
```

Output

```
25
25
```

Part	Meaning
{ }	A lambda is always written inside curly braces
n: Int	The parameter(s)
->	Separates the parameters from the body
n * n	The body. The **last line** is the result (no return keyword)

2. More lambda examples

Different lambdas

```
fun main() {
    val sayHello = { println("Hello!") }                  // no parameters
    val add = { a: Int, b: Int -> a + b }                  // two parameters
    val describe = { name: String, age: Int ->
        val status = if (age >= 18) "adult" else "minor"   // many lines
        "$name is an $status"                              // last line = result
    }

    sayHello()
    println(add(3, 4))
    println(describe("Anjana", 20))
}
```

Output

```
Hello!
7
Anjana is an adult
```

3. Function types

Lambdas have types too. A function type shows the parameter types and the return type:

Function type	Meaning
() -> Unit	No input, returns nothing
(Int) -> Int	Takes an Int, returns an Int
(String, Int) -> String	Takes a String and an Int, returns a String
(Int) -> Boolean	Takes an Int, returns true or false

Writing the type explicitly

```
fun main() {
    val double: (Int) -> Int = { n -> n * 2 }
    val isAdult: (Int) -> Boolean = { age -> age >= 18 }
    val greet: () -> Unit = { println("Welcome!") }

    println(double(21))
    println(isAdult(16))
    greet()
}
```

Output

```
42
false
Welcome!
```

4. it: the shortcut for one parameter

If a lambda has **exactly one** parameter, you don't have to name it. Kotlin calls it **it**.

Using it

```
fun main() {
    val triple: (Int) -> Int = { it * 3 }
    val shout: (String) -> String = { it.uppercase() + "!" }

    println(triple(5))
    println(shout("kotlin"))
}
```

Output

```
15
KOTLIN!
```

5. Higher-order functions

A **higher-order function** is a function that **takes another function as a parameter** (or returns one). This lets you write one function and change its behaviour by passing different lambdas.

Passing a lambda to a function

```
fun calculate(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
    return operation(a, b)
}

fun main() {
    println(calculate(10, 5, { x, y -> x + y }))
    println(calculate(10, 5, { x, y -> x - y }))
    println(calculate(10, 5, { x, y -> x * y }))
}
```

Output

```
15
5
50
```

6. Trailing lambda syntax

If the **last** parameter is a function, you can move the lambda **outside** the brackets. This is the style you'll see everywhere in Kotlin and Jetpack Compose.

Trailing lambdas

```
fun repeatAction(times: Int, action: (Int) -> Unit) {
    for (i in 1..times) {
        action(i)
    }
}

fun main() {
    // Lambda inside the brackets
    repeatAction(2, { println("Inside: round $it") })

    // Trailing lambda: same thing, easier to read
    repeatAction(3) {
        println("Trailing: round $it")
    }
}
```

Output

```
Inside: round 1
Inside: round 2
Trailing: round 1
Trailing: round 2
Trailing: round 3
```

Tip: You've already used trailing lambdas without knowing it: **repeat(3) { ... }** and **x?.let { ... }**!

7. Function references with ::

You can also pass a normal named function using **::functionName**.

Function references

```
fun isEven(n: Int): Boolean = n % 2 == 0

fun checkAll(numbers: List<Int>, test: (Int) -> Boolean) {
    for (n in numbers) {
        println("$n -> ${test(n)}")
    }
}

fun main() {
    checkAll(listOf(3, 8, 11), ::isEven)
}
```

Output

```
3 -> false
8 -> true
11 -> false
```

8. Complete example: a discount calculator

Different discount rules

```
fun finalPrice(price: Double, discountRule: (Double) -> Double): Double {
    return price - discountRule(price)
}

fun main() {
    val noDiscount: (Double) -> Double = { 0.0 }
    val tenPercent: (Double) -> Double = { it * 0.10 }
    val flat200: (Double) -> Double = { if (it >= 1000) 200.0 else 0.0 }

    val price = 1500.0
    println("No discount : Rs ${finalPrice(price, noDiscount)}")
    println("10% off     : Rs ${finalPrice(price, tenPercent)}")
    println("Flat Rs 200 : Rs ${finalPrice(price, flat200)}")
    println("Half price  : Rs ${finalPrice(price) { it / 2 }}")
}
```

Output

```
No discount : Rs 1500.0
10% off     : Rs 1350.0
Flat Rs 200 : Rs 1300.0
Half price  : Rs 750.0
```

Practice Exercises

1. Write a lambda **cube** that returns n * n * n, and print cube(4).

2. Write a lambda **isPositive: (Int) -> Boolean** using **it**. Test it with 5, -3 and 0.

3. Write a lambda that takes a first name and a last name and returns the full name with a space between them.

4. Write a higher-order function **applyTwice(n: Int, f: (Int) -> Int): Int** that applies f two times. Test it with { it + 3 } and { it * 2 } on the number 5.

5. Write **operateOnList(numbers: List<Int>, f: (Int) -> Int)** that prints f applied to each number. Call it with a trailing lambda that squares the numbers.

6. Write a normal function **isVowel(c: Char): Boolean** and pass it with :: to a function that counts the matching letters in "education".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is a lambda?

Q2. In { a: Int, b: Int -> a + b }, what is the result?

Q3. When can you use it?

Q4. What is a higher-order function?

Q5. What is a trailing lambda?

Q6. What does (String) -> Int mean?

WEEK 2 • DAY 5

Collection Operations

Today's goal: Process lists with forEach, filter, map, count, any/all, find, sortedBy, sumOf and groupBy.

1. Why collection operations?

Yesterday you learned lambdas. Today you'll use them with Kotlin's built-in collection functions. They replace long loops with **one clear line**. Compare these two ways of getting the even numbers:

Loop vs filter

```
fun main() {
    val numbers = listOf(1, 2, 3, 4, 5, 6, 7, 8)

    // Old way: loop
    val evens1 = mutableListOf<Int>()
    for (n in numbers) {
        if (n % 2 == 0) evens1.add(n)
    }
    println(evens1)

    // Kotlin way: filter
    val evens2 = numbers.filter { it % 2 == 0 }
    println(evens2)
}
```

Output

```
[2, 4, 6, 8]
[2, 4, 6, 8]
```

2. forEach: do something with each item

forEach

```
fun main() {
    val names = listOf("Anu", "Ravi", "Meera")
    names.forEach { println("Hello, $it") }

    names.forEachIndexed { index, name ->
        println("${index + 1}. $name")
    }
}
```

Output

```
Hello, Anu
Hello, Ravi
Hello, Meera
1. Anu
2. Ravi
3. Meera
```

3. filter: keep only the items that match

filter

```
fun main() {
    val marks = listOf(45, 82, 37, 91, 60, 28)

    val passed = marks.filter { it >= 40 }
    val failed = marks.filter { it < 40 }
    val distinction = marks.filter { it >= 80 }

    println("Passed: $passed")
    println("Failed: $failed")
    println("Distinction: $distinction")

    val names = listOf("Anjana", "Ravi", "Arun", "Meera")
    println(names.filter { it.startsWith("A") })
}
```

Output

```
Passed: [45, 82, 91, 60]
Failed: [37, 28]
Distinction: [82, 91]
[Anjana, Arun]
```

4. map: change every item

map creates a **new list** by changing every item. (This is the map *function*, which is different from the Map *collection* you learned on Day 3.)

map

```
fun main() {
    val prices = listOf(100, 250, 80)
    val withTax = prices.map { it * 1.18 }
    println(withTax)

    val names = listOf("anjana", "ravi")
    println(names.map { it.uppercase() })
    println(names.map { it.length })

    val marks = listOf(45, 82, 37)
    println(marks.map { if (it >= 40) "Pass" else "Fail" })
}
```

Output

```
[118.0, 295.0, 94.39999999999999]
[ANJANA, RAVI]
[6, 4]
[Pass, Pass, Fail]
```

Careful: Notice **94.39999999999999** instead of 94.4. Computers store decimals in binary, so tiny rounding errors happen. To show 2 decimal places, use `String.format("%.2f", value)`.

5. Checking and counting

count, any, all, none, find

```
fun main() {
    val marks = listOf(45, 82, 37, 91, 60)

    println(marks.count { it >= 40 })     // how many passed
    println(marks.any { it > 90 })        // is at least one above 90?
    println(marks.all { it >= 40 })       // did everyone pass?
    println(marks.none { it == 100 })     // nobody scored 100?
    println(marks.find { it < 40 })       // first one below 40
    println(marks.find { it > 95 })       // none: null
}
```

Output

```
4
true
false
true
37
null
```

6. Sorting, summing and grouping

These work best with lists of pairs or objects. A **Pair** holds two values; **first** and **second** read them. (Next week you'll learn data classes, which are even better.)

sortedBy, maxBy, sumOf

```
fun main() {
    val students = listOf("Anu" to 78, "Ravi" to 92, "Meera" to 85)

    println(students.sortedBy { it.second })            // lowest first
    println(students.sortedByDescending { it.second })  // highest first
    println(students.maxBy { it.second })               // topper
    println(students.sumOf { it.second })               // total marks

    val words = listOf("kiwi", "banana", "fig", "mango")
    println(words.sortedBy { it.length })
}
```

Output

```
[(Anu, 78), (Meera, 85), (Ravi, 92)]
[(Ravi, 92), (Meera, 85), (Anu, 78)]
(Ravi, 92)
255
[fig, kiwi, mango, banana]
```

groupBy and partition

```
fun main() {
    val names = listOf("Anu", "Ravi", "Arun", "Meera", "Rahul")
    println(names.groupBy { it.first() })

    val marks = listOf(45, 82, 37, 91, 28)
    val (pass, fail) = marks.partition { it >= 40 }
    println("Pass: $pass, Fail: $fail")
}
```

Output

```
{A=[Anu, Arun], R=[Ravi, Rahul], M=[Meera]}
Pass: [45, 82, 91], Fail: [37, 28]
```

7. Chaining operations

Each operation returns a new collection, so you can **chain** them one after another. Read the chain from top to bottom.

Chaining

```
fun main() {
    val marks = listOf(45, 82, 37, 91, 60, 28, 75)

    val result = marks
        .filter { it >= 40 }        // [45, 82, 91, 60, 75]
        .sortedDescending()         // [91, 82, 75, 60, 45]
        .take(3)                    // [91, 82, 75]
        .map { "$it marks" }

    println(result)

    val total = (1..10).filter { it % 2 == 1 }.sumOf { it * it }
    println("Sum of squares of odd numbers: $total")
}
```

Output

```
[91 marks, 82 marks, 75 marks]
Sum of squares of odd numbers: 165
```

8. Operations on maps

filter and map on a Map

```
fun main() {
    val prices = mapOf("Rice" to 60, "Milk" to 28, "Ghee" to 650, "Eggs" to 72)

    println(prices.filter { it.value > 50 })
    println(prices.map { "${it.key}: Rs ${it.value}" })
    println(prices.maxBy { it.value }.key)
    println(prices.values.sum())
}
```

Output

```
{Rice=60, Ghee=650, Eggs=72}
[Rice: Rs 60, Milk: Rs 28, Ghee: Rs 650, Eggs: Rs 72]
Ghee
810
```

9. Cheat sheet

Function	What it returns	Example
forEach { }	Nothing (just does something)	list.forEach { println(it) }
filter { }	List of matching items	list.filter { it > 0 }
map { }	List of changed items	list.map { it * 2 }
count { }	Int	list.count { it >= 40 }
any / all / none { }	Boolean	list.any { it > 90 }
find { }	First match, or null	list.find { it < 40 }
sortedBy { }	Sorted list	list.sortedBy { it.length }
maxBy / minBy { }	The item with the biggest/smallest value	list.maxBy { it.second }
sumOf { }	Sum	list.sumOf { it.second }
groupBy { }	Map of key to list	list.groupBy { it.first() }
partition { }	Pair of two lists	list.partition { it >= 40 }

Practice Exercises

1. From numbers 1 to 20, use filter to get the numbers divisible by 3.

2. Use map to convert a list of Celsius temperatures [0, 25, 37, 100] to Fahrenheit (C * 9 / 5 + 32, using Doubles).

3. Given names ["anjana", "ravi", "meera"], create a list where each name starts with a capital letter. (Hint: `it.replaceFirstChar { c -> c.uppercase() }`)

4. Given marks [56, 89, 34, 72, 95, 41], print: how many scored above 70, whether anyone failed (below 40), and the first mark above 90.

5. Given a list of pairs (product to price), print the cheapest product and the total price using minBy and sumOf.

6. Given words ["apple", "avocado", "banana", "blueberry", "cherry"], group them by their first letter.

7. Using a chain, from [12, 5, 8, 21, 3, 17, 10] get the even numbers, multiply each by 10, and print the result sorted from highest to lowest.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between filter and map?

Q2. What does find return if nothing matches?

Q3. What is the difference between any and all?

Q4. What does groupBy return?

Q5. Do these operations change the original list?

WEEK 2 • DAY 6

Weekly Project: Student Marks Manager

Today's goal: Combine null safety, maps, lambdas and collection operations to build a complete program.

1. What we're building

A **Student Marks Manager** that can:

- Add or update a student's marks (only 0 to 100 allowed)
- Remove a student
- Show all students, sorted from highest to lowest, with grades
- Show a class report: average, topper, failed students and grade counts

We store the data in a **mutable map** of name to marks, because each name should appear only once.

2. Plan the functions first

Function	Job	Week 2 skills used
addStudent(name, marks)	Validate and save marks	Map, ranges (!in)
removeStudent(name)	Remove if present	Map, null check
gradeOf(marks)	Turn marks into a grade	when
showAll()	Print everyone, sorted	sortedByDescending, forEachIndexed
showReport()	Print statistics	average, maxBy, filter, groupBy, mapValues

Tip: Professional developers always **plan first**: decide what data you store and which functions you need, then write the code one function at a time, testing each one.

3. The code (Version 1: Playground)

students is declared **outside** main (at the top level) so that every function can use it.

Student Marks Manager v1

```
val students = mutableMapOf<String, Int>()

fun addStudent(name: String, marks: Int) {
    if (marks !in 0..100) {
        println("Marks must be between 0 and 100")
        return
    }
    students[name] = marks
    println("Saved: $name ($marks)")
}

fun removeStudent(name: String) {
    if (students.remove(name) != null) println("Removed: $name")
    else println("No student called $name")
}

fun gradeOf(marks: Int): String = when {
    marks >= 90 -> "A+"
```

```
    marks >= 80 -> "A"
    marks >= 70 -> "B"
    marks >= 40 -> "C"
    else -> "F"
}

fun showAll() {
    if (students.isEmpty()) {
        println("No students yet")
        return
    }
    println("--- All students (highest first) ---")
    students.entries
        .sortedByDescending { it.value }
        .forEachIndexed { i, entry ->
            println("${i + 1}. ${entry.key}: ${entry.value} (${gradeOf(entry.value)})")
        }
}

fun showReport() {
    if (students.isEmpty()) {
        println("No students yet")
        return
    }
    val average = students.values.average()
    val topper = students.maxBy { it.value }
    val failed = students.filter { it.value < 40 }.keys
    val gradeCount = students.values.groupBy { gradeOf(it) }.mapValues { it.value.size }

    println("--- Class report ---")
    println("Students : ${students.size}")
    println("Average  : ${String.format("%.1f", average)}")
    println("Topper   : ${topper.key} (${topper.value})")
    println("Failed   : ${if (failed.isEmpty()) "None" else failed.joinToString()}")
    println("Grades   : $gradeCount")
}

fun main() {
    addStudent("Anjana", 92)
    addStudent("Ravi", 78)
    addStudent("Meera", 85)
    addStudent("John", 35)
    addStudent("Arun", 150)       // invalid
    addStudent("Ravi", 81)        // update existing
    removeStudent("Zara")         // not found
    println()
    showAll()
    println()
    showReport()
}
```

Output

```
Saved: Anjana (92)
Saved: Ravi (78)
Saved: Meera (85)
Saved: John (35)
Marks must be between 0 and 100
Saved: Ravi (81)
No student called Zara

--- All students (highest first) ---
1. Anjana: 92 (A+)
```

```
2. Meera: 85 (A)
3. Ravi: 81 (A)
4. John: 35 (F)

--- Class report ---
Students : 4
Average  : 73.3
Topper   : Anjana (92)
Failed   : John
Grades   : {A+=1, A=2, F=1}
```

4. Understand the tricky lines

Code	What it does
marks !in 0..100	true if marks is **not** between 0 and 100
return (in a Unit function)	Stops the function early
students.remove(name) != null	remove returns the old value, or null if the name wasn't there
students.entries	All key-value pairs, so we can sort them
students.maxBy { it.value }	The entry with the highest marks
filter { it.value < 40 }.keys	Names of the students who failed
groupBy { gradeOf(it) }	Groups the marks by grade, e.g. {A=[85, 81], ...}
.mapValues { it.value.size }	Replaces each list with its size, e.g. {A=2, ...}
String.format("%.1f", average)	Shows the average with 1 decimal place

5. Version 2: interactive menu (IntelliJ)

Keep all the functions from Version 1 and replace main with this menu. It uses **toIntOrNull()** and **?:** from Day 1, so wrong input never crashes the program.

Student Marks Manager v2 (main only) (run in IntelliJ: input typed by the user is shown in the output)

```
fun main() {
    while (true) {
        println("1. Add/Update  2. Remove  3. Show all  4. Report  5. Exit")
        print("Choose: ")
        when (readln().trim()) {
            "1" -> {
                print("Name: ")
                val name = readln().trim()
                print("Marks: ")
                val marks = readln().toIntOrNull()
                if (name.isEmpty() || marks == null) println("Invalid input")
                else addStudent(name, marks)
            }
            "2" -> {
                print("Name to remove: ")
                removeStudent(readln().trim())
            }
            "3" -> showAll()
            "4" -> showReport()
            "5" -> {
                println("Goodbye!")
                return
            }
            else -> println("Please choose 1 to 5")
        }
        println()
```

```
    }
}
```

Output

```
1. Add/Update  2. Remove  3. Show all  4. Report  5. Exit
Choose: 1
Name: Anjana
Marks: 92
Saved: Anjana (92)

1. Add/Update  2. Remove  3. Show all  4. Report  5. Exit
Choose: 1
Name: Ravi
Marks: abc
Invalid input

1. Add/Update  2. Remove  3. Show all  4. Report  5. Exit
Choose: 1
Name: Ravi
Marks: 78
Saved: Ravi (78)

1. Add/Update  2. Remove  3. Show all  4. Report  5. Exit
Choose: 4
--- Class report ---
Students : 2
Average  : 85.0
Topper   : Anjana (92)
Failed   : None
Grades   : {A+=1, B=1}

1. Add/Update  2. Remove  3. Show all  4. Report  5. Exit
Choose: 5
Goodbye!
```

6. Project challenges

- Add a menu option **Search**: ask for a name and print its marks, or "Not found" (use ?: or let).
- Add an option to show only the students with marks **above a number** the user types (use filter).
- Store **three subjects** per student with `mutableMapOf<String, List<Int>>()` and use each student's average for the grade.
- Upload the project to your **kotlin-practice** GitHub repository.

Week 2 checklist

I can...	Done?
Explain null and declare nullable types with ?	[ ]
Use ?., ?: and let, and explain why !! is dangerous	[ ]
Create and change lists with listOf and mutableListOf	[ ]
Use sets for unique values and maps for key-value data	[ ]
Write lambdas, including ones that use it	[ ]
Write a higher-order function and call it with a trailing lambda	[ ]
Use filter, map, forEach, count, any, all and find	[ ]
Use sortedBy, maxBy, sumOf and groupBy	[ ]
Chain several operations together	[ ]
Build the Student Marks Manager	[ ]

Quick Quiz

Q1. Why did we use a Map instead of a List to store the students?

Q2. What does students.remove(name) return when the name doesn't exist?

Q3. Why does Version 2 use toIntOrNull() instead of toInt()?

Q4. Which function finds the topper?

Q5. What does mapValues { it.value.size } do?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Null Safety

Exercise 1

```
fun main() {
    val middleName: String? = null
    if (middleName == null) {
        println("No middle name")
    } else {
        println(middleName)
    }
}
```

Output

```
No middle name
```

Exercise 2

```
fun main() {
    val a: String? = "Kotlin"
    val b: String? = null
    println(a?.length ?: 0)
    println(b?.length ?: 0)
}
```

Output

```
6
0
```

Exercise 3

```
fun main() {
    println("42".toIntOrNull() ?: -1)
    println("hello".toIntOrNull() ?: -1)
    println("7".toIntOrNull() ?: -1)
}
```

Output

```
42
-1
7
```

Exercise 4

```
fun getGrade(marks: Int?): String {
    if (marks == null) return "Absent"
    return if (marks >= 40) "Pass" else "Fail"
}

fun main() {
    println(getGrade(null))
    println(getGrade(75))
    println(getGrade(20))
}
```

Output

```
Absent
Pass
Fail
```

Exercise 5

```
fun main() {
    val phone1: String? = "98470 12345"
    val phone2: String? = null
    phone1?.let { println("Phone: $it") }
    phone2?.let { println("Phone: $it") }
}
```

Output

```
Phone: 98470 12345
```

Exercise 6

```
fun findCapital(state: String): String? {
    return when (state) {
        "Kerala" -> "Thiruvananthapuram"
        "Tamil Nadu" -> "Chennai"
        "Karnataka" -> "Bengaluru"
        else -> null
    }
}

fun main() {
    println(findCapital("Kerala") ?: "Unknown")
    println(findCapital("Goa") ?: "Unknown")
}
```

Output

```
Thiruvananthapuram
Unknown
```

Quiz answers

Q1. No value.

Q2. Add ? after the type, for example String?.

Q3. null. It does not crash.

Q4. 10.

Q5. It crashes the program if the value is null.

Q6. it

Day 2: Lists: Storing Many Values

Exercise 1

```
fun main() {
    val movies = listOf("Premam", "Drishyam", "Bangalore Days", "Kumbalangi Nights", "Minnal Murali")
    println(movies)
    println(movies.size)
    println(movies.first())
    println(movies.last())
}
```

Output

```
[Premam, Drishyam, Bangalore Days, Kumbalangi Nights, Minnal Murali]
5
Premam
Minnal Murali
```

Exercise 2

```
fun main() {
    val numbers = listOf(12, 7, 25, 3, 18, 9)
    println("Sum: ${numbers.sum()}")
    println("Average: ${numbers.average()}")
    println("Max: ${numbers.max()}")
    println("Min: ${numbers.min()}")
}
```

Output

```
Sum: 74
Average: 12.333333333333334
Max: 25
Min: 3
```

Exercise 3

```
fun main() {
    val friends = mutableListOf<String>()
    friends.add("Anu")
    friends.add("Ravi")
    friends.add("Meera")
    friends.add("John")
    friends.remove("Ravi")
    for ((i, name) in friends.withIndex()) {
        println("${i + 1}. $name")
    }
}
```

Output

```
1. Anu
2. Meera
3. John
```

Exercise 4

```
fun main() {
    val squares = mutableListOf<Int>()
    for (i in 1..10) {
        squares.add(i * i)
    }
    println(squares)
}
```

Output

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Exercise 5

```
fun main() {
    val marks = listOf(45, 82, 37, 91, 60)
    var passed = 0
    for (m in marks) {
        if (m >= 40) passed++
    }
    println("Passed: $passed")
}
```

Output

```
Passed: 4
```

Exercise 6

```
fun secondLargest(numbers: List<Int>): Int {
    return numbers.distinct().sortedDescending()[1]
}

fun main() {
    println(secondLargest(listOf(4, 9, 2, 9, 7)))
}
```

Output

```
7
```

Quiz answers

Q1. listOf is read-only; mutableListOf lets you add, remove and change items.

Q2. 0

Q3. list.size

Q4. The program crashes with IndexOutOfBoundsException. Use getOrNull() to be safe.

Q5. No. It returns a new sorted list.

Q6. mutableListOf<Int>()

Day 3: Sets & Maps

Exercise 1

```
fun main() {
    val unique = listOf(3, 7, 3, 9, 7, 1).toSet()
    println(unique)
    println(unique.size)
}
```

Output

```
[3, 7, 9, 1]
4
```

Exercise 2

```
fun main() {
    val hobbies = mutableSetOf("Reading", "Music")
    println(hobbies.add("Music"))
    println(hobbies)
}
```

Output

```
false
[Reading, Music]
```

Exercise 3

```
fun main() {
    val fruits = mapOf("Mango" to 120, "Banana" to 50, "Apple" to 180, "Grapes" to 90)
    for ((fruit, price) in fruits) {
        println("$fruit: Rs $price")
    }
    println("Total: Rs ${fruits.values.sum()}")
}
```

Output

```
Mango: Rs 120
Banana: Rs 50
Apple: Rs 180
Grapes: Rs 90
Total: Rs 440
```

Exercise 4

```
fun main() {
    val marks = mutableMapOf("Anu" to 78, "Ravi" to 65, "Meera" to 88)
    marks["John"] = 72
    marks["Ravi"] = 70
    marks.remove("Anu")
    println(marks)
}
```

Output

```
{Ravi=70, Meera=88, John=72}
```

Exercise 5

```
fun main() {
    val counts = mutableMapOf<Char, Int>()
    for (ch in "programming") {
        counts[ch] = (counts[ch] ?: 0) + 1
    }
    println(counts)
}
```

Output

```
{p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```

Exercise 6

```
fun getMonthDays(month: String): Int {
    val days = mapOf(
        "January" to 31, "February" to 28, "March" to 31, "April" to 30,
        "May" to 31, "June" to 30, "July" to 31, "August" to 31,
        "September" to 30, "October" to 31, "November" to 30, "December" to 31
    )
    return days[month] ?: 0
}

fun main() {
    println(getMonthDays("February"))
    println(getMonthDays("Funday"))
}
```

Output

```
28
0
```

Quiz answers

Q1. Nothing. It is not added, and add() returns false.

Q2. key to value, for example "Kerala" to "Thiruvananthapuram".

Q3. null

Q4. map[key] = value

Q5. list.toSet()

Q6. No. Keys are unique. Setting an existing key replaces its value.

Day 4: Lambdas & Higher-Order Functions

Exercise 1

```
fun main() {
    val cube = { n: Int -> n * n * n }
    println(cube(4))
}
```

Output

```
64
```

Exercise 2

```
fun main() {
    val isPositive: (Int) -> Boolean = { it > 0 }
    println(isPositive(5))
    println(isPositive(-3))
    println(isPositive(0))
}
```

Output

```
true
false
false
```

Exercise 3

```
fun main() {
    val fullName = { first: String, last: String -> "$first $last" }
    println(fullName("Anjana", "Menon"))
}
```

Output

```
Anjana Menon
```

Exercise 4

```
fun applyTwice(n: Int, f: (Int) -> Int): Int {
    return f(f(n))
}

fun main() {
    println(applyTwice(5) { it + 3 })
    println(applyTwice(5) { it * 2 })
}
```

Output

```
11
20
```

Exercise 5

```
fun operateOnList(numbers: List<Int>, f: (Int) -> Int) {
    for (n in numbers) {
        println(f(n))
    }
}

fun main() {
    operateOnList(listOf(2, 3, 4)) { it * it }
}
```

Output

```
4
9
16
```

Exercise 6

```
fun isVowel(c: Char): Boolean = c in "aeiou"

fun countMatching(text: String, test: (Char) -> Boolean): Int {
    var count = 0
    for (ch in text) {
        if (test(ch)) count++
    }
    return count
}

fun main() {
    println(countMatching("education", ::isVowel))
}
```

Output

```
5
```

Quiz answers

Q1. A function without a name that you can store in a variable or pass to another function.

Q2. a + b. The last expression is the result.

Q3. When the lambda has exactly one parameter.

Q4. A function that takes a function as a parameter or returns one.

Q5. A lambda written after the brackets when it is the last parameter, for example repeat(3) { ... }.

Q6. A function that takes a String and returns an Int.

Day 5: Collection Operations

Exercise 1

```
fun main() {
    println((1..20).filter { it % 3 == 0 })
}
```

Output

```
[3, 6, 9, 12, 15, 18]
```

Exercise 2

```
fun main() {
    val celsius = listOf(0.0, 25.0, 37.0, 100.0)
    println(celsius.map { it * 9 / 5 + 32 })
}
```

Output

```
[32.0, 77.0, 98.6, 212.0]
```

Exercise 3

```
fun main() {
    val names = listOf("anjana", "ravi", "meera")
    println(names.map { it.replaceFirstChar { c -> c.uppercase() } })
}
```

Output

```
[Anjana, Ravi, Meera]
```

Exercise 4

```
fun main() {
    val marks = listOf(56, 89, 34, 72, 95, 41)
    println("Above 70: ${marks.count { it > 70 }}")
    println("Anyone failed: ${marks.any { it < 40 }}")
    println("First above 90: ${marks.find { it > 90 }}")
}
```

Output

```
Above 70: 3
Anyone failed: true
First above 90: 95
```

Exercise 5

```
fun main() {
    val products = listOf("Pen" to 10, "Notebook" to 45, "Bag" to 650, "Ruler" to 15)
    println("Cheapest: ${products.minBy { it.second }.first}")
    println("Total: Rs ${products.sumOf { it.second }}")
}
```

Output

```
Cheapest: Pen
Total: Rs 720
```

Exercise 6

```
fun main() {
    val words = listOf("apple", "avocado", "banana", "blueberry", "cherry")
    println(words.groupBy { it.first() })
}
```

Output

```
{a=[apple, avocado], b=[banana, blueberry], c=[cherry]}
```

Exercise 7

```
fun main() {
    val result = listOf(12, 5, 8, 21, 3, 17, 10)
        .filter { it % 2 == 0 }
        .map { it * 10 }
        .sortedDescending()
    println(result)
}
```

Output

```
[120, 100, 80]
```

Quiz answers

Q1. filter keeps some items unchanged; map changes every item.

Q2. null

Q3. any is true if at least one item matches; all is true only if every item matches.

Q4. A Map from each key to the list of items with that key.

Q5. No. They return new collections.

Day 6: Weekly Project: Student Marks Manager

Quiz answers

Q1. Each name must be unique, and we want to look up marks by name.

Q2. null

Q3. So the program doesn't crash when the user types text instead of a number.

Q4. maxBy { it.value }

Q5. It replaces each value (a list) with that list's size.

> **Next:** When you finish this week, tell your mentor and ask for the **Week 3** material. Great work, Anjana!